

[INSTITUTE / UNIVERSITY NAME]

MINI PROJECT REPORT

FASAL

Crop Yield Management and Field History Dashboard

Submitted in partial fulfilment of the requirements for the award of the degree of

Bachelor of Technology

[Programme / Department]

Submitted by

[Student Name]

[Roll Number]

Under the guidance of

[Project Guide Name]

[Designation]



DEPARTMENT OF [DEPARTMENT NAME]

SESSION: [ACADEMIC SESSION]

Public website: aayushhhho6.github.io/Cropwise-miniproject

CERTIFICATE

This is to certify that the project report entitled “**Fasal: Crop Yield Management and Field History Dashboard**” is a record of the project work carried out by **[Student Name]** (Roll No. **[Roll Number]**) of **[Programme / Department]**, **[Institute Name]**, during **[Academic Session]**, under the guidance of **[Project Guide Name and Designation]**.

To the best of our knowledge, this project report fulfils the academic requirements prescribed for the mini project.

[Project Guide Name]
Project Guide

[Project Coordinator Name]
Project Coordinator

[Head of Department Name]
Head of Department

Date: _____
Place: _____

Completion note: Replace the bracketed fields with verified academic details and obtain the required signatures before submitting this report to an institution.

DECLARATION

I, **[Student Name]**, declare that the project entitled “**Fasal: Crop Yield Management and Field History Dashboard**” is a description of work undertaken by me as part of the mini project for **[Programme / Department]** at **[Institute Name]**, under the guidance of **[Project Guide Name]**.

I further declare that this report has been prepared for academic purposes, that implementation details have been described to the best of my knowledge, and that sources used have been acknowledged in the references. Any estimates produced by the software are educational and are not professional agronomic advice.

Date: _____

Place: _____

[Student Name]

Candidate Signature

ACKNOWLEDGEMENT

I would like to express my gratitude to **[Project Guide Name]** and the faculty of **[Department / Institute]** for their guidance and academic support. I also thank my classmates, friends, and family for their encouragement during the design, implementation, and review of this project. This acknowledgement is an editable template and should be personalised before formal submission.

ABSTRACT

Farm planning relies on observations about crop choice, field size, weather, soil moisture, pH, organic matter, and nutrients. These observations are often kept informally, making it difficult to compare field conditions or maintain a simple record of estimates. Fasal is a responsive crop-yield management dashboard that brings these inputs and a lightweight crop-specific estimation model into one interface.

The public application is delivered as a static website through GitHub Pages. A user selects one of five supported crops—rice, wheat, maize, cotton, or sugarcane—and enters field measurements. Browser JavaScript validates the inputs, calculates a weighted soil-health score and estimated yield, displays a general recommendation, and saves submitted reports in the browser's local storage. Reports can be reviewed, deleted, cleared, or exported as CSV. The layout adapts to desktop, tablet, and smartphone screens.

The repository also retains an optional Flask development backend. That implementation uses SQLite for persistent records and NumPy for the same type of crop-profile yield calculation, with HTML templates and JSON endpoints. It is distinct from the Pages deployment: GitHub Pages does not execute Flask or provide a shared database. As a result, reports entered on the public static site remain on that device and browser and do not synchronise across users or devices.

The application demonstrates a complete, accessible workflow for educational field reporting, while making the limits of its simplified model and storage explicit. Estimates should not replace laboratory soil testing or advice from a qualified agronomist.

Keywords: crop yield estimation, farm dashboard, soil health, JavaScript, browser storage, GitHub Pages, Flask, SQLite, NumPy.

TABLE OF CONTENTS

1. Chapter 1: Introduction
 1. Project Background
 2. Problem Statement
 3. Proposed Solution
 4. Objectives
 5. Scope and Limitations
 6. Methodology
 7. Expected Outcomes
2. Chapter 2: Background and Related Work
 1. Introduction
 2. Farm Records and Yield Estimates
 3. Web Delivery Models
 4. Limitations and Design Considerations
 5. Summary
3. Chapter 3: System Design and Architecture
 1. Requirements
 2. Architecture Overview
 3. Data Model
 4. Estimation Workflow
 5. Security and Privacy
4. Chapter 4: Implementation
 1. Technology Stack
 2. Static Front-End
 3. Yield and Health Calculation
 4. Browser History and Export
 5. Optional Flask and SQLite Backend
 6. GitHub Pages Deployment
5. Chapter 5: Testing and Validation
 1. Objectives and Strategy
 2. Validation Cases
 3. Deployment Verification
 4. Known Limitations
6. Chapter 6: Results, Conclusion, and Future Scope
7. References
8. Appendix: Running the Optional Flask Backend

This is a digitally generated report. Page numbers may differ depending on the browser, paper size, and print settings used when saving it as PDF.

INTRODUCTION

1.1 Project Background

Digital tools can help farmers organise field observations and compare crop scenarios. However, a practical field log does not need to begin with a complex farm-management platform. A focused interface that records a small set of conditions, explains its estimate, and keeps a usable history can support learning and basic planning. Fasal was developed as a mini project to demonstrate that workflow in a responsive web application.

Fasal accepts crop type, farm area, temperature, soil moisture, soil pH, organic matter, nitrogen, and phosphorus. It calculates an indicative health score and yield from crop-specific reference profiles. The public version is a static website accessible through the repository's GitHub Pages URL on modern phones and computers.

1.2 Problem Statement

Field measurements and yield expectations may be noted in separate places or estimated informally. Without a simple record interface, users may find it difficult to validate readings, compare reports over time, and keep estimates associated with a crop and field area. A solution must also be accessible on common screen sizes without requiring visitors to install an application.

1.3 Proposed Solution

Fasal provides a browser-based dashboard with a structured field form, input-range validation, crop-specific estimate, recommendation text, and a recent-report history. Users can export report history in CSV format. GitHub Pages hosts the static web interface; an optional Flask/SQLite version is retained in the repository for local backend development and demonstrations.

1.4 Objectives

1. Provide a simple, responsive interface for entering field and crop observations.
2. Validate numeric inputs against defined ranges before calculating an estimate.
3. Calculate a weighted field-health score and crop-specific yield estimate.
4. Keep a reviewable report history and provide CSV export.
5. Publish the static website through GitHub Pages for access by URL.
6. Document storage, model, and deployment limitations accurately.

1.5 Scope and Limitations

The static website supports five crops and seven numeric field measurements. It provides an educational score, per-acre estimate, total estimate, and rule-based recommendation. Reports are kept in browser local storage, which is scoped to the browser profile and device. There is no user account, shared cloud database, weather service, sensor integration, or synchronisation.

The reference values and weighted formula are illustrative rather than calibrated against local yield datasets. The tool does not account for cultivar, season, irrigation management, pests, disease, seed quality, or local agronomic conditions. Its estimates must not be used as a guarantee of production, financial return, or fertiliser prescription.

1.6 Methodology

1. **Requirement identification:** define the crop inputs, estimate outputs, browser history, and mobile-friendly page structure.
2. **Interface design:** adapt the existing green dashboard design and organise the workflow into form, estimate, recommendations, and history.
3. **Model implementation:** encode crop-specific target profiles and a weighted score, with input ranges enforced at the form.
4. **Persistence and export:** save reports to local storage and generate a CSV download in the browser.
5. **Publishing:** serve the standalone HTML, CSS, and JavaScript from the repository root on GitHub Pages.
6. **Validation:** check document/assets, form-to-script element references, browser storage workflow, and the deployed Pages response.

1.7 Expected Outcomes

The expected outcome is a publicly accessible, responsive application that calculates illustrative crop estimates, retains a local history, and clearly explains that data is not shared between devices. The repository's optional Flask implementation demonstrates how a server-backed version could be built and tested locally.

BACKGROUND AND RELATED WORK

2.1 Introduction

This chapter frames the project in relation to common approaches to farm record keeping and lightweight web delivery. It is a design-oriented review, not a systematic survey or a claim that Fasal has been validated in field trials.

2.2 Farm Records and Yield Estimates

Field logs can preserve measurements that help users compare conditions across observations. A digital form improves consistency by naming each input, showing units, and applying permitted ranges. Crop models can range from simple rule-based calculations to calibrated statistical and process-based models. Fasal intentionally uses a transparent, weighted heuristic suitable for demonstrating application flow; it is not trained on a yield dataset and should not be interpreted as a production-grade agronomic model.

2.3 Web Delivery Models

A static website consists of files that a web server can deliver without executing application code on the server. This makes it suitable for GitHub Pages and broadly accessible through a normal URL. Browser storage can preserve small amounts of user data locally, but it is not a substitute for authenticated, central persistence. A dynamic Flask application can accept requests and use a SQL database, but it requires a separate server host; GitHub Pages does not run Python processes.

2.4 Design Comparison

Approach	Strength	Trade-off	Fasal use
Paper or informal records	Simple and works without devices	Harder to validate, search, and calculate consistently	Digital form and browser history provide structured records
Static browser application	Low operational overhead; easy public hosting	Local data is not shared or synchronised	Used by the GitHub Pages website
Server-backed application	Can provide shared database and API	Requires hosting, operations, and access-control design	Optional Flask/SQLite implementation in the source repository
Data-calibrated crop model	Can reflect validated local observations	Requires suitable data, calibration, and domain evaluation	Future extension; current model is a transparent heuristic

2.5 Gaps and Design Considerations

For this project, the principal design challenge is to avoid implying capabilities that the deployment does not provide. The published website therefore identifies browser-local storage and describes calculations as estimates. A shared, multi-device history would require a hosted backend, database, authentication, data protection, and operational ownership; those components are outside the deployed Pages application.

2.6 Summary

Fasal combines a lightweight web interface with structured inputs and an explainable calculation. The choice of static deployment makes the website easy to publish, while local-only persistence and the non-calibrated model remain explicit limitations.

SYSTEM DESIGN AND ARCHITECTURE

3.1 Functional Requirements

- Select a supported crop and enter the seven field measurements.
- Reject missing, non-numeric, or out-of-range numeric values.
- Calculate a crop-specific health score, fertility indicator, and yield estimate.
- Display recommendations and nutrient status from the provided inputs.
- Save reports locally, display recent history, delete a report, clear history, and export CSV.
- Render usefully across smartphone, tablet, and desktop viewport sizes.

3.2 Architecture Overview

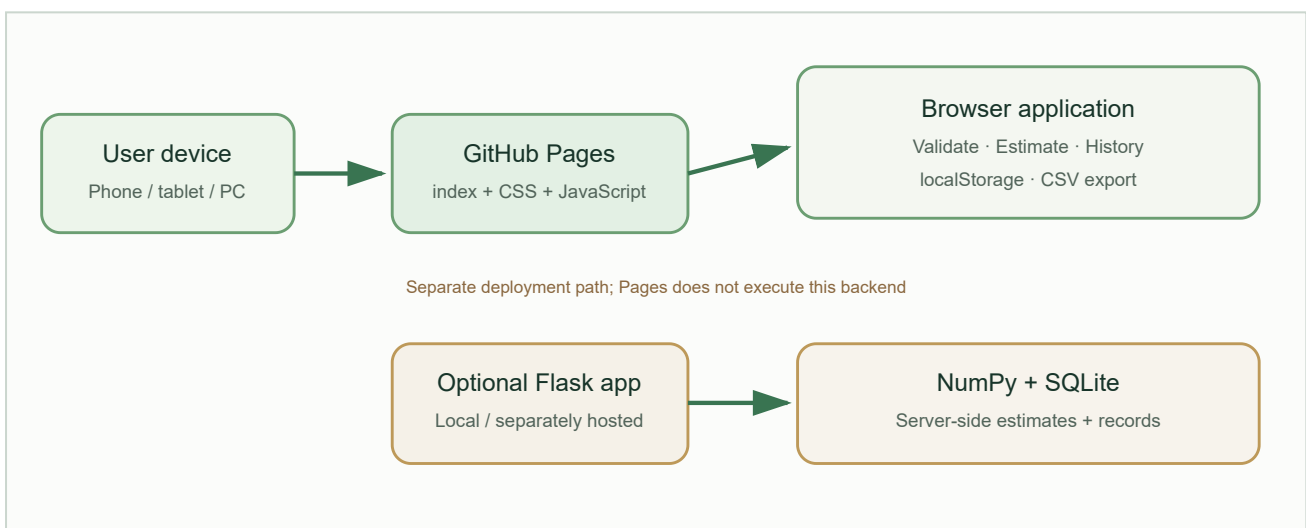


Figure 3.1: Published static architecture and optional server-backed development architecture.

3.3 Data Model

The static application stores a JSON array under a versioned local-storage key. Each report includes a unique identifier, creation time, field input values, and derived estimate. The optional Flask database stores one row per report in the `crop_records` SQLite table.

Field	Unit	Accepted range	Purpose
Crop	Category	Rice, wheat, maize, cotton, sugarcane	Selects target profile and base yield
Area	Acres	0.1–10,000	Scales per-acre yield to field total
Temperature	°C	0–60	Compared with crop profile target
Soil moisture	%	0–100	Compared with crop profile target
Soil pH	pH	0–14	Contributes to soil score and guidance
Organic matter	%	0–15	Contributes to soil score and fertility
Nitrogen	kg/ha	0–300	Compared with crop profile target
Phosphorus	kg/ha	0–150	Compared with crop profile target

3.4 Estimation Workflow

1. The form reads the selected crop and numeric inputs.
2. Browser-native required/minimum/maximum checks and JavaScript guards validate the values.
3. The crop profile supplies ideal temperature, moisture, nitrogen, phosphorus, and base yield.
4. Component scores are clamped from 0 to 100 and combined using fixed weights.
5. The interface displays a score, yield, fertility, nutrient statuses, and rule-based recommendation.
6. On submission, the input and estimate are stored as a local browser report and added to history.

3.5 Security and Privacy

The Pages application does not require an account and sends report history to no application server; browser storage remains local to the browser origin. Users should avoid entering sensitive personal information and should export or back up data they need to keep. Clearing browser data or changing devices can remove access to the saved history. The separate Flask backend has no user authentication and is intended for local development unless security controls and a production database are added before deployment.

IMPLEMENTATION

4.1 Technology Stack

Component	Technology	Role
Public interface	HTML5, CSS3	Responsive dashboard and field form
Public calculation	Browser JavaScript	Input validation, score/yield, recommendations, local history and CSV
Public hosting	GitHub Pages	Serves static files from the main branch root
Optional backend	Python, Flask	Routes, template rendering, request validation and local API
Backend calculations	NumPy	Clipping and weighted score calculations
Backend storage	SQLite	Persistent crop report table for the local Flask app

4.2 Front-End Implementation

The root `index.html` is a standalone document so GitHub Pages can serve it without a build step or server-side template engine. The existing dashboard visual design is shared through `static/styles.css`. Responsive CSS rearranges the sidebar, cards, form, and history table at tablet and mobile widths. The calculator and history are semantic form/section structures with labels and status messages for usability.

4.3 Crop Profiles and Health Score

Each supported crop profile contains target values and an illustrative yield baseline. The six score components represent temperature, moisture, pH, organic matter, nitrogen, and phosphorus. For a target value, closeness is mapped into a bounded component score. Organic matter is scaled to a three-percent reference and capped. The weighted total is rounded to an integer from 0 to 100.

```

S = round(0.18T + 0.17M + 0.18H + 0.14O + 0.18N + 0.15P)
T = clamp(100 - 7 × |temperature - ideal temperature|, 0, 100)
M = clamp(100 - 2.4 × |moisture - ideal moisture|, 0, 100)
H = clamp(100 - 30 × |pH - 6.7|, 0, 100)

```

```
O = clamp(organic matter / 3 × 100, 0, 100)
N, P = clamp(100 - 80 × |measured - ideal| / ideal, 0, 100)
```

The derived fertility indicator and yield use the following equations:

```
Fertility = round(clamp(0.98 × S + 2 × organic matter, 0, 100))
Yield per acre = round to 0.1 (base yield × (0.6 + S / 250))
Total yield = round to 0.1 (yield per acre × area in acres)
```

The app's target profiles are configuration constants in the source; they are not verified recommendations. The reported total uses tonnes and the profile baseline is treated as tonnes per acre for display. Such simplified assumptions limit the scientific meaning of the output.

4.4 Recommendations and History

The recommendation logic checks moisture first, then nitrogen, phosphorus, and pH, and displays the first applicable general prompt. The history view presents the crop, local date, score, and yield. A report can be individually deleted; all history can be cleared after confirmation. The CSV export serialises saved reports for a spreadsheet tool. Storage exceptions are reported to the user rather than treated as successful saves.

4.5 Optional Flask and SQLite Backend

The repository retains `app.py`, `schema.sql`, and Jinja templates for local backend experimentation. Database initialization creates the SQLite report table and date index. The form route validates values and inserts them with parameterised SQL. `/api/records` returns saved records and estimates; `/api/estimate` validates and returns a non-persistent estimate. This backend is not invoked by the public GitHub Pages site.

4.6 GitHub Pages Deployment

The public website is hosted at <https://aayushhhho6.github.io/Cropwise-miniproject/>. GitHub Pages is configured to deploy from the `main` branch at the repository root. It serves the root `index.html` together with `static/app.js` and `static/styles.css`. Changes merged to the publishing branch trigger a Pages build.

TESTING AND VALIDATION

5.1 Objectives

Testing checks that the public files can be served, the form is connected to its script, numeric validation protects the estimate flow, local history operations behave coherently, and the published site responds from GitHub Pages.

5.2 Testing Strategy

Test area	Verification	Result recorded
Static document wiring	Check JavaScript element IDs against the root HTML and confirm referenced CSS/script files exist.	Passed during implementation.
Static HTTP smoke test	Serve the static project locally and request the home page, JavaScript, and CSS.	All three returned HTTP 200 with expected markers.
Public deployment	Request the published Pages home page and deployed assets; inspect the Pages build and source settings.	Build status “built”; home page and assets returned HTTP 200.
Estimate and validation	Inspect required fields, range checks, supported crop profile lookup, and estimate formulas.	Implemented in the public browser app; should also be exercised in target browsers.
Persistence and export	Save, reload, delete, clear, and export browser-local records.	Implemented; behaviour depends on browser storage permissions and should be confirmed on target devices.
Backend smoke checks	Exercise Flask dashboard, invalid submission, SQLite insert, records API, and estimate API.	Passed in an isolated-database smoke run during backend implementation.

5.3 Input Validation

The interface combines HTML numeric attributes with JavaScript checks. The range boundaries are inclusive except farm area, whose minimum is 0.1 acre. Unsupported crops are rejected. Reports are not appended when browser storage cannot save the data; a visible message explains the failure.

5.4 Deployment Validation

The configured Pages source was verified as `main` and `/(root)`. The latest Pages build completed with status `built`. The deployed root page and the JavaScript and CSS assets returned successful HTTP responses and contained expected dashboard content.

5.5 Limitations of Validation

These checks establish basic wiring and deployment availability, not agronomic accuracy, accessibility conformance, cross-browser compatibility, load capacity, or field usability. No controlled crop trials or independent user-acceptance study are claimed. Such evidence would be required before making operational farming recommendations.

5.6 Conclusion

The performed smoke checks support that the public static site is deployed and its key assets are reachable. Functional acceptance across a wider range of browsers, screen readers, storage settings, and farm use cases remains future work.

RESULTS, CONCLUSION, AND FUTURE SCOPE

6.1 Results

The project delivers a public responsive dashboard with five crop profiles, a validated field-entry form, a crop-health estimate, yield and nutrient indicators, rule-based guidance, local report history, record deletion, clear-history control, and CSV export. The public static build is deployed at the GitHub Pages URL and verified to return the page and assets successfully.

A separate Flask/SQLite implementation is retained for local backend learning and provides persistent records and JSON estimate/history routes when run on a Python host. It is not the runtime behind the GitHub Pages URL. The browser and backend implementations have similar objectives but different storage locations and deployment requirements.

6.2 Conclusion

Fasal demonstrates an end-to-end crop reporting workflow using accessible web technologies. Hosting the front end on GitHub Pages provides a public URL without requiring users to install software. Browser-local storage keeps basic history without a server, while the report explains that this history is device-specific. The project fulfils its educational software objectives but does not establish the agronomic validity of its illustrative yield estimates.

6.3 Future Scope

1. Evaluate the estimation model against trustworthy, local crop-yield datasets and seek review from qualified agronomists.
2. Introduce a secured, hosted backend and managed SQL database for authenticated cross-device synchronisation.
3. Add fields such as location, crop variety, season, sowing date, irrigation, and actual harvest to support meaningful comparisons.
4. Support import, backup, and restore workflows in addition to CSV export.
5. Improve language accessibility and conduct usability testing with intended users.
6. Complete formal accessibility, browser compatibility, privacy, and security reviews before wider operational use.
7. Integrate weather or sensor data only with reliable data sources, user consent, and clearly documented uncertainty.

BIBLIOGRAPHY

REFERENCES

1. GitHub Docs, "What is GitHub Pages?" <https://docs.github.com/en/pages/getting-started-with-github-pages/what-is-github-pages>. Accessed 27 September 2026.
2. Mozilla Developer Network, "Window: localStorage property." <https://developer.mozilla.org/en-US/docs/Web/API/Window/localStorage>. Accessed 27 September 2026.
3. Flask Documentation, "Flask Documentation." <https://flask.palletsprojects.com/>. Accessed 27 September 2026.
4. NumPy Developers, "NumPy documentation." <https://numpy.org/doc/>. Accessed 27 September 2026.
5. SQLite Consortium, "SQLite Documentation." <https://www.sqlite.org/docs.html>. Accessed 27 September 2026.
6. Fasal project source code and deployment: <https://github.com/aayushhhho6/Cropwise-miniproject>; live site: <https://aayushhhho6.github.io/Cropwise-miniproject/>. Accessed 27 September 2026.

RUNNING THE OPTIONAL FLASK BACKEND

The public website is already hosted on GitHub Pages. The steps below run the optional Flask development implementation locally; they do not deploy it to a public server.

A.1 Requirements

- Python 3.10 or newer.
- Flask and NumPy installed in the active Python environment.
- A terminal with Bash, PowerShell, or equivalent commands.

A.2 Commands

```
python -m venv .venv
# Linux/macOS Bash
source .venv/bin/activate
# Windows PowerShell: .venv\Scripts\Activate.ps1
python -m pip install Flask numpy
python app.py
```

Open `http://127.0.0.1:5000`. The optional Flask app initializes an SQLite database at `data/fasal.db` by default. Deployment of that backend requires a Python-capable hosting platform and a separately managed database strategy.

For submitting this report: Open this HTML file in a browser, choose Print, select “Save as PDF,” and use A4 paper. Fill in the bracketed cover/certificate details first. Add actual screenshots or institution-specific pages only if required by the project coordinator.